COMP3121 Notes

Ronnie Low

What is an algorithm?
Algorithms are a set of precisely defined steps are executed based off mechanical steps - there is no intelligence involved, just follow the steps given.

Types of algorithms
1. Sequential deterministic algorithms:
- Only one step can be executed at a time
- No element of randomness

2. Parallel algorithms:
- Multiple steps can be ran at a time
- Probably not covered in 3121

3. Randomised algorithms:
- Utilises random values
- Used in quantum computing
- Probably not covered in 3121

Why prove algorithms?
Sometimes, its not obvious if an algorithm does the things its advertised through common sense:
- Will your algorithm terminate in a timely manner?
- Can you prove that the algorithm won't infinitely loop?
- Is the time complexity as advertised?

Proofs are needed for these cases, most of the time they're the **only** way to know if the algorithm does its job.

Stable matching example
There are *n* hospitals and *n* doctors. Each hospital wants to hire exactly one doctor. Each doctor and hospital submit a list of preferences ranking which hospital they want to go to and which doctor they want.

How do we design an algorithm where there is no hospital doctor pair (*h, d*) that are dissatisfied with their result (stable):

$$\text{for pair } p = (h, d') \ \& \ p' = (h', d)$$

- h prefers d over d'
- h' prefers d' ovr d

This implies that both hospitals and doctors have a preferred swap to make (unstable match)

Stable matching algorithm
While there is a free hospital, pick a free hospital h and offer a job to the highest ranking doctor d on its preference list.

If d doesn't have a job yet, d always accepts the job.

else if d is already in a pair $(h'd)$:
- if h is higher ranked on the preference list for d, a new pair (h, d) is created and the previous pair (h', d) is deleted.
- if h is ranked lower, then the offer is ignored.

How many ways to match hospital-doctors?
$n!$, first doctor has n choices, next doctor has $n-1$ choices, so on and so forth.

Will a stable match always exist?
Yes, but it is not obvious - we need to use a proof:

note that during the while loop:
- for a doctor, the hospital they go to will always be higher on their ranks.
- a hospital will be paired with doctors of decreasing ranks.

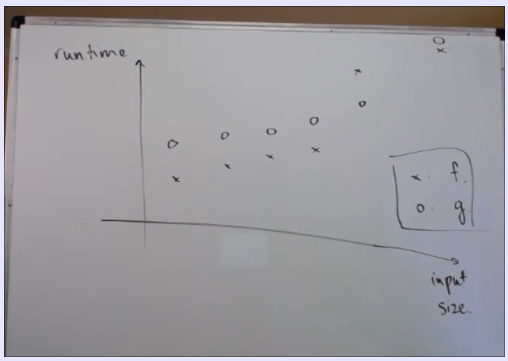
If we assume that the ranking is not stable: $p = (h, d)$ and $p' = (h', d')$ such that h prefers d' and h' prefers d.

Since h prefers d' to d, then it must have made an offer to d' before offering to d, but since h is paired with d, then d' would need to have rejected h because tehy were already at a hospital they prefer to h. This proves that its impossible to have d' prefer h' if h is ranked higher.

Rates of growth (Recap)
There are statements like:
- *Heap sort is faster than bubble sort.*
- *Linear search is slower than binary search.*

We want our language to be more precise; when is it faster, will it be not the case?

If we were to compare two algorithms, issues may occur!
- Outliers! We can see in the graph above that although g is generally slower, the fifth point has an outlier where f is slower. How do we describe the changes/differences?
- Implementation differences: what if f' is slower than both g and f, but f' has a different implementation?



We mostly perfer to talk in terms of long term behaviour (asymtopics)

For instance, if the size of an input doubles, the function could double, triple, or even quadruple!

Big O Notation

We say that $f(n) = O(g(n))$ if:

There are positive constants C and N such that $0 \leq f(n) \leq C \times g(n)$ for all $n \geq N$.

What does that acutally mean? lets take a look at some examples.

Is $f(n) = O(g(n))$ for:

$f(n) = n + 5$, $g(n) = n^2 - 1$

For this to hold true, there needs to be a value n and C such that every single value of $f(n) \leq g(n)$. **Is this true?**

Well, given a sufficiently large n, yes! Lets prove it:
Lets assign $C = 1$

$$f(n) = n + 5 < g(n) = n^2 - 1 \tag{1}$$
$$0 < n^2 - n - 6 \tag{2}$$
$$0 < (n-3)(n+2) \tag{3}$$
$$\therefore n \text{ is true for } n > 3 \text{ or } n < -2 \tag{4}$$

this implies that for values $C = 1$ and every value $n > 3$, this equation holds true!
In more human terms, we **need** to have a value C that is multiplied with $g(n)$, and there **needs** to exist a value n such that every value greater than n will satisfy

$$0 \leq f(n) \leq C \times g(n).$$

In doing so, we can say that the for $f(n)$ (which is the function of input size vs time) has a time complexity of $O(g(n))$ (keep in mind $g(n)$ will just be an equation that $f(n)$ will never exceed.)